

Prototype Design Pattern — Bilingual Study Document (English + Hindi)

Inspired by [algomaster.io/learn/lld/prototype](https://www.algomaster.io/learn/lld/prototype) — concepts and structure preserved, written in original wording with self-authored PHP examples

Study Companion · Updated July 2026

Prototype Design Pattern

Note on sourcing / स्रोत पर टिप्पणी: AlgoMaster.io is a paid interview-prep course, and its code samples sit behind a subscription. This document follows the same topic order and teaching structure as their Prototype lesson (the encapsulation/dependency problem → shallow vs deep copy → an enemy-spawning game example → a registry → an email-template example), but every explanation below is written fresh in my own words, and all PHP code is original — written to teach the same ideas, not copied from the source.

AlgoMaster.io एक पेड (paid) इंटरव्यू-प्रेप कोर्स है, और इसके कोड सैंपल सब्सक्रिप्शन (subscription) के पीछे हैं। यह दस्तावेज़ उनके Prototype पाठ जैसा ही विषय-क्रम और टीचिंग स्ट्रक्चर फ़ॉलो करता है (एनकैप्सुलेशन/डिपेंडेंसी की समस्या → शैलो बनाम डीप कॉपी → एक एनिमी-स्पॉनिंग गेम का उदाहरण → एक रजिस्ट्री → एक ईमेल-टेम्पलेट उदाहरण), लेकिन नीचे दी गई हर व्याख्या मेरे अपने शब्दों में ताज़ा लिखी गई है, और सारा PHP कोड ओरिजिनल (original) है — वही विचार सिखाने के लिए लिखा गया, स्रोत से कॉपी नहीं किया गया।

Overview

The **Prototype Design Pattern** is a **creational design pattern** that creates new objects by **cloning an existing, already-configured object**, instead of building each one from scratch through a constructor.

प्रोटोटाइप डिज़ाइन पैटर्न (Prototype Design Pattern) एक **क्रिएशनल डिज़ाइन पैटर्न (creational design pattern)** है, जो नए ऑब्जेक्ट्स को कंस्ट्रक्टर के ज़रिए शुरू से बनाने के बजाय, एक मौजूदा, पहले से कॉन्फ़िगर किए गए ऑब्जेक्ट को **क्लोन (clone)** करके बनाता है।

It earns its keep in three situations: when constructing a fresh object is slow or resource-heavy, when you want to avoid re-running the same complicated setup logic every time, and when you need many objects that are almost identical except for a few tweaked fields.

यह तीन स्थितियों में खास तौर पर काम आता है: जब एक नया ऑब्जेक्ट बनाना धीमा या रिसोर्स-हैवी (resource-heavy) हो, जब आप हर बार वही जटिल (complicated) सेटअप लॉजिक दोबारा चलाने से बचना चाहते हों, और जब आपको ऐसे कई ऑब्जेक्ट्स चाहिए हों जो लगभग एक जैसे हों, बस कुछ फ़िल्ड्स में थोड़ा-सा बदलाव हो।

Instead of repeating that setup, you configure one "template" object once, and produce every future instance by cloning it — which keeps the results consistent and cuts down on repeated code.

बार-बार वही सेटअप दोहराने के बजाय, आप एक "टेम्पलेट (template)" ऑब्जेक्ट को एक बार कॉन्फ़िगर करते हैं, और आगे के हर instance को उसे क्लोन करके बनाते हैं — इससे नतीजे एक जैसे (consistent) बने रहते हैं और दोहराए गए कोड की मात्रा घट जाती है।

1. The Challenge of Cloning Objects

Suppose you already have an object and simply want an exact duplicate of it. The obvious first instinct is: create a new object of the same class, then manually copy every field over. That sounds simple — but it runs into trouble fast.

मान लीजिए आपके पास पहले से एक ऑब्जेक्ट है और आप बस उसकी एक सटीक (exact) डुप्लीकेट (duplicate) चाहते हैं। पहला और सबसे स्वाभाविक (obvious) खयाल यही आता है: उसी क्लास का एक नया ऑब्जेक्ट बनाओ, फिर हर फ़ील्ड को हाथ से कॉपी कर दो। सुनने में यह आसान लगता है — लेकिन जल्दी ही यह मुश्किल में पड़ जाता है।

Problem 1: Encapsulation Gets in the Way

That manual-copy approach only works if every field is publicly reachable. But a well-designed class deliberately keeps many of its fields **private**, hidden behind encapsulation — and outside code simply cannot read them to copy them over, without breaking the whole point of encapsulation in the first place.

समस्या 1: एनकैप्सुलेशन आड़े आता है

यह मैनुअल-कॉपी (manual-copy) तरीका तभी काम करता है जब हर फ़ील्ड सार्वजनिक रूप से (publicly) पहुँच योग्य हो। लेकिन एक अच्छी तरह डिज़ाइन की गई क्लास जान-बूझकर अपनी कई फ़ील्ड्स को **प्राइवेट (private)** रखती है, एनकैप्सुलेशन (encapsulation) के पीछे छुपाकर — और बाहरी कोड उन्हें कॉपी करने के लिए पढ़ ही नहीं सकता, बिना एनकैप्सुलेशन के पूरे मक़सद को तोड़े।

Problem 2: Class-Level Dependency

Even where every field were accessible, you'd still need to know the object's exact concrete class to construct a matching copy. That ties your copying logic tightly to that one class — which fights the Open/Closed Principle, makes the code brittle if the class changes, and gets awkward the moment polymorphism is involved and you're really only holding a reference to some interface.

समस्या 2: क्लास-लेवल डिपेंडेंसी

चाहे हर फ़ील्ड एक्सेसिबल (accessible) भी हो, फिर भी आपको मेल खाती (matching) कॉपी बनाने के लिए ऑब्जेक्ट की असली (concrete) क्लास पता होनी ज़रूरी होगी। इससे आपका कॉपी करने वाला लॉजिक उस एक क्लास से कसकर बंध (tightly tied) जाता है — जो ओपन/क्लोज़्ड प्रिंसिपल (Open/Closed Principle) के खिलाफ़ जाता है, क्लास बदलने पर कोड को नाज़ुक (brittle) बना देता है, और जैसे ही पॉलिमॉर्फ़िज़्म (polymorphism) शामिल होता है और आपके पास असल में सिर्फ़ किसी इंटरफ़ेस का रेफ़रेंस होता है, तब यह और भी अजीब हो जाता है।

Problem 3: Interface-Only Contexts

A lot of real code never even sees the concrete class — it only ever works through an interface. If a piece of code receives something typed as a generic `Shape`, it genuinely has no way to

know whether that's a `Circle`, a `Square`, or some class defined in a completely different module. It can't call `new` on a class name it doesn't know, so it's stuck — unless the object itself is willing to do the copying.

समस्या 3: सिर्फ इंटरफ़ेस वाले संदर्भ

बहुत सारा असली (real) कोड कभी असली क्लास देखता ही नहीं — वह सिर्फ एक इंटरफ़ेस के ज़रिए काम करता है। अगर किसी कोड को कोई ऐसी चीज़ मिलती है जिसकी टाइप एक जनरल `Shape` है, तो उसे सच में पता नहीं होता कि यह `Circle` है, `Square` है, या किसी बिल्कुल अलग मॉड्यूल (module) में डिफ़ाइन की गई कोई क्लास है। वह किसी ऐसे क्लास नाम पर `new` कॉल नहीं कर सकता जो उसे पता ही नहीं — इसलिए वह अटक जाता है, जब तक कि ऑब्जेक्ट खुद कॉपी करने के लिए तैयार न हो।

The Better Way: Let the Object Clone Itself

The fix is to flip who's responsible. Instead of external code trying to peek inside an object and rebuild it, the object itself exposes a `clone()` method and knows how to produce its own copy. This keeps encapsulation intact, removes the need for the caller to know the concrete class at all, and scales cleanly as new types are added.

बेहतर तरीका: ऑब्जेक्ट को खुद क्लोन करने दें

समाधान यह है कि ज़िम्मेदारी (responsibility) को पलट दिया जाए। बाहरी कोड द्वारा ऑब्जेक्ट के अंदर झाँकने और उसे दोबारा बनाने की कोशिश करने के बजाय, ऑब्जेक्ट खुद एक `clone()` मेथड एक्सपोज़ करता है और जानता है कि अपनी खुद की कॉपी कैसे बनानी है। इससे एनकैप्सुलेशन बरकरार रहता है, कॉलर (caller) को असली क्लास जानने की ज़रूरत पूरी तरह खत्म हो जाती है, और नई टाइप्स जोड़े जाने पर भी यह साफ़-सुथरे तरीके से स्केल (scale) होता है।

Shallow Copy vs Deep Copy

This is the single most important idea to get right when implementing Prototype — getting it wrong produces bugs that show up intermittently and are painful to trace back.

शैलो कॉपी बनाम डीप कॉपी

Prototype को सही तरीके से इम्प्लीमेंट करते समय यह सबसे ज़रूरी विचार है — इसे ग़लत करने पर ऐसे बग्स (bugs) पैदा होते हैं जो कभी-कभार दिखते हैं और जिन्हें ढूँढ़ना बेहद मुश्किल होता है।

A **shallow copy** duplicates the object's own memory, but any field that's a reference to another object — a list, a map, a nested object — still points at the *same* underlying object as the original. Primitive fields (numbers, booleans, immutable strings) are safe, because they're copied by value; reference fields are the danger zone.

एक **शैलो कॉपी (shallow copy)** ऑब्जेक्ट की अपनी मेमोरी (memory) को डुप्लीकेट करती है, लेकिन कोई भी ऐसी फ़ील्ड जो किसी दूसरे ऑब्जेक्ट का रेफ़रेंस (reference) है — जैसे कोई लिस्ट, मैप, या नेस्टेड (nested) ऑब्जेक्ट — वह अब भी मूल (original) की उसी अंतर्निहित (underlying) ऑब्जेक्ट की ओर इशारा करती रहती है। प्रिमिटिव फ़ील्ड्स (numbers, booleans, immutable strings) सुरक्षित होती हैं, क्योंकि वे वैल्यू (value) के हिसाब से कॉपी होती हैं; रेफ़रेंस फ़ील्ड्स ही खतरे का क्षेत्र (danger zone) हैं।

A **deep copy** goes further: it duplicates the object, and every object it refers to, and every object *those* refer to, recursively, until the clone shares nothing mutable with the original. Whenever a prototype holds a mutable reference field, this is the copy strategy it needs.

एक **डीप कॉपी (deep copy)** इससे आगे जाती है: यह ऑब्जेक्ट को, और उसके द्वारा रेफर (refer) किए गए हर ऑब्जेक्ट को, और उन ऑब्जेक्ट्स द्वारा रेफर किए गए हर ऑब्जेक्ट को — रिकर्सिवली (recursively) — तब तक डुप्लीकेट करती है जब तक क्लोन और मूल के बीच कोई भी म्यूटेबल (mutable) चीज़ साझा (shared) न रह जाए। जब भी किसी prototype में कोई म्यूटेबल रेफरेंस फ़ील्ड होती है, उसे यही कॉपी रणनीति (copy strategy) चाहिए होती है।

2. The Problem: Spawning Enemies in a Game

Picture a 2D shooter where enemies appear constantly during play. Say there are three enemy types, each with its own baseline stats: a **Basic Enemy** with low health and slow speed for early levels, an **Armored Enemy** with high health and medium speed that's tougher to bring down, and a **Flying Enemy** with medium health and fast speed used for surprise attacks. Each type carries a bundle of properties — health, speed, whether it's armored, its weapon, and its sprite/appearance.

एक 2D शूटर (shooter) गेम की कल्पना कीजिए जिसमें खेल के दौरान लगातार दुश्मन (enemies) दिखाई देते हैं। मान लीजिए तीन एनिमी टाइप्स (enemy types) हैं, हर एक की अपनी बेसलाइन (baseline) स्टैट्स (stats) के साथ: एक **Basic Enemy**, जिसकी हेल्थ (health) कम और स्पीड (speed) धीमी है, शुरुआती लेवल्स के लिए; एक **Armored Enemy**, जिसकी हेल्थ ज्यादा और स्पीड मध्यम है, जिसे हराना मुश्किल है; और एक **Flying Enemy**, जिसकी हेल्थ मध्यम और स्पीड तेज़ है, सरप्राइज़ अटैक (surprise attack) के लिए इस्तेमाल होता है। हर टाइप कई प्रॉपर्टीज़ (properties) के साथ आता है — हेल्थ, स्पीड, वह आर्मर्ड (armored) है या नहीं, उसका हथियार (weapon), और उसका स्प्राइट/दिखावट (sprite/appearance)।

If every time you need a Flying Enemy you write a fresh constructor call setting each of those properties by hand, and you repeat that across dozens of spawn points in the codebase, four problems creep in quickly: the same setup code gets duplicated everywhere, default values are scattered so a single balance change means hunting down every copy, it's easy to forget a property or set it wrong, and the spawn logic itself gets cluttered with construction detail that has nothing to do with actual gameplay.

अगर हर बार जब आपको एक Flying Enemy चाहिए, आप हाथ से हर प्रॉपर्टी सेट करते हुए एक नया कंस्ट्रक्टर कॉल लिखते हैं, और यह कोडबेस (codebase) में दर्जनों स्पॉन पॉइंट्स (spawn points) पर दोहराते हैं, तो जल्दी ही चार समस्याएँ पैदा होती हैं: वही सेटअप कोड हर जगह दोहराया जाता है, डिफ़ॉल्ट वैल्यूज़ (default values) बिखरी (scattered) रहती हैं, इसलिए एक भी बैलेंस (balance) बदलाव करने के लिए हर कॉपी ढूँढ़नी पड़ती है, कोई प्रॉपर्टी भूल जाना या ग़लत सेट कर देना आसान हो जाता है, और स्पॉन लॉजिक खुद कंस्ट्रक्शन (construction) की बारीकियों से अस्त-व्यस्त (cluttered) हो जाता है, जिनका असली गेमप्ले से कोई लेना-देना नहीं है।

What's needed is one place where each enemy type's defaults are configured once, plus a cheap, reliable way to hand out new instances built from that configuration.

जो चाहिए वह है एक ऐसी जगह जहाँ हर एनिमी टाइप के डिफ़ॉल्ट्स को एक ही बार कॉन्फ़िगर किया जाए, साथ ही उस कॉन्फ़िगरेशन से बने नए instances को सस्ते (cheap) और भरोसेमंद (reliable) तरीक़े से बाँटने का एक तरीक़ा।

3. The Prototype Design Pattern — Definition

Put simply: the Prototype pattern lets you define the kind of object you need through one fully-configured example, and produce every further instance by cloning that example rather than constructing it anew.

सीधे शब्दों में: Prototype पैटर्न आपको उस तरह के ऑब्जेक्ट को एक पूरी तरह कॉन्फ़िगर किए गए उदाहरण (example) के ज़रिए परिभाषित (define) करने देता है, और आगे के हर instance को उस उदाहरण को क्लोन करके बनाता है, न कि उसे नए सिरे से बनाकर।

Two ideas sit at the core of it: **self-cloning**, meaning the object itself owns the logic for copying itself, so no outside code needs to understand its internals; and **decoupled creation**, meaning the client only ever talks to a common interface with a `clone()` method, and never needs to know which concrete class it's actually cloning.

इसके मूल में दो विचार हैं: **सेल्फ-क्लोनिंग (self-cloning)**, यानी ऑब्जेक्ट खुद अपनी कॉपी बनाने का लॉजिक रखता है, इसलिए किसी बाहरी कोड को उसका आंतरिक (internal) ढाँचा समझने की ज़रूरत नहीं पड़ती; और **डीकपल्ड क्रिएशन (decoupled creation)**, यानी क्लाइंट सिर्फ़ एक कॉमन इंटरफ़ेस से `clone()` मेथड के ज़रिए बात करता है, और उसे कभी यह जानने की ज़रूरत नहीं पड़ती कि वह असल में किस असली (concrete) क्लास को क्लोन कर रहा है।

Participants

Prototype (interface). Declares the `clone()` method that every cloneable class must implement — this is the contract that lets a client copy an object without knowing its concrete type.

Prototype (इंटरफ़ेस)। `clone()` मेथड को डिक्लेयर करता है, जिसे हर क्लोनेबल (cloneable) क्लास को इम्प्लीमेंट करना ज़रूरी है — यही वह कॉन्ट्रैक्ट (contract) है जो क्लाइंट को बिना असली टाइप जाने किसी ऑब्जेक्ट को कॉपी करने देता है।

ConcretePrototype. Implements `clone()` and does the actual copying of its own fields into the new instance — shallow or deep, depending on what each field holds.

ConcretePrototype। `clone()` को इम्प्लीमेंट करती है और अपनी फ़िल्ड्स को नए instance में असल में कॉपी करती है — शैलो या डीप, यह इस बात पर निर्भर करता है कि हर फ़िल्ड क्या रखती है।

Client. Holds a reference to a prototype and calls `clone()` whenever it needs a new object, then optionally tweaks the returned copy.

Client। एक prototype का रेफ़रेंस रखता है और जब भी उसे नया ऑब्जेक्ट चाहिए `clone()` कॉल करता है, फिर वैकल्पिक रूप से (optionally) मिली हुई कॉपी में बदलाव करता है।

Prototype Registry (optional). A lookup table — keyed by string, enum, or similar — of pre-built prototypes. Callers ask the registry for an object by key, and it always hands back a fresh clone, never the stored original.

Prototype Registry (वैकल्पिक)। पहले से बने प्रोटोटाइप्स की एक लुकअप टेबल (lookup table) — जिसे स्ट्रिंग, enum, या ऐसे ही किसी कुंजी (key) से इंडेक्स किया जाता है। कॉलर्स रजिस्ट्री से किसी key के ज़रिए ऑब्जेक्ट माँगते हैं, और यह हमेशा एक ताज़ा (fresh) क्लोन लौटाती है, कभी भी स्टोर किया गया मूल (original) नहीं।

4. How It Works — Five Steps

Step 1 — Build the prototype instances. Configure one fully-initialized "template" object per type you'll need — for the game, one properly-tuned `FlyingEnemy`, one `ArmoredEnemy`, and so on.

चरण 1 — प्रोटोटाइप इंस्टेंसेज़ बनाएँ। हर ज़रूरी टाइप के लिए एक पूरी तरह इनिशियलाइज़ किया गया "टेम्पलेट" ऑब्जेक्ट कॉन्फ़िगर करें — गेम के लिए, एक सही ढंग से ट्यून (tuned) किया `FlyingEnemy`, एक `ArmoredEnemy`, वगैरह।

Step 2 — Register the prototypes (optional). Store each template in a registry under a meaningful key, like `"flying"` or `"armored"` — this centralizes configuration and makes adding new types trivial.

चरण 2 — प्रोटोटाइप्स को रजिस्टर करें (वैकल्पिक)। हर टेम्पलेट को एक सार्थक (meaningful) key के तहत रजिस्ट्री में स्टोर करें, जैसे `"flying"` या `"armored"` — इससे कॉन्फ़िगरेशन केंद्रीकृत (centralized) हो जाता है और नई टाइप्स जोड़ना बेहद आसान हो जाता है।

Step 3 — The client asks for a clone. When the game needs a new enemy, it asks the registry for one by key; the registry finds the matching prototype and calls its `clone()` method.

चरण 3 — क्लाइंट एक क्लोन माँगता है। जब गेम को नया एनिमी चाहिए, वह रजिस्ट्री से key के ज़रिए एक माँगता है; रजिस्ट्री मेल खाता प्रोटोटाइप ढूँढ़ती है और उसका `clone()` मेथड कॉल करती है।

Step 4 — The prototype copies itself. A brand-new instance is created and every field is transferred into it — the clone is an independent object in memory carrying the same values.

चरण 4 — प्रोटोटाइप खुद को कॉपी करता है। एक बिल्कुल नया instance बनाया जाता है और हर फ़ील्ड उसमें ट्रांसफ़र (transfer) की जाती है — क्लोन मेमोरी में एक स्वतंत्र (independent) ऑब्जेक्ट है, जो वही वैल्यूज़ रखता है।

Step 5 — The client customizes the clone. The caller is free to tweak the copy — say, lowering health to spawn a "weakened" variant — with zero effect on the original prototype, which stays untouched and ready for the next clone.

चरण 5 — क्लाइंट क्लोन को कस्टमाइज़ करता है। कॉलर कॉपी में बदलाव करने के लिए स्वतंत्र है — जैसे, एक "कमज़ोर (weakened)" वैरिएंट स्पॉन करने के लिए हेल्थ घटाना — मूल प्रोटोटाइप पर इसका कोई असर नहीं पड़ता, जो अछूता (untouched) रहता है और अगले क्लोन के लिए तैयार रहता है।

The client never calls a constructor directly — it always goes through the registry, gets back a ready-to-use object, and optionally adjusts it.

क्लाइंट कभी भी सीधे कंस्ट्रक्टर कॉल नहीं करता — वह हमेशा रजिस्ट्री से होकर गुज़रता है, एक इस्तेमाल के लिए तैयार ऑब्जेक्ट पाता है, और वैकल्पिक रूप से उसे एडजस्ट (adjust) करता है।

5. Implementing Prototype (in PHP)

The implementation below is original code written for this study document, illustrating the same enemy-spawning scenario in four steps: define the contract, write a shallow clone, upgrade it to a deep clone once a mutable field appears, then add a registry and client code.

नीचे दिया गया इम्प्लीमेंटेशन इसी अध्ययन दस्तावेज़ के लिए लिखा गया ओरिजिनल कोड है, जो उसी एनिमी-स्पॉनिंग परिदृश्य (scenario) को चार चरणों में दिखाता है: कॉन्ट्रैक्ट डिफ़ाइन करना, एक शैलो क्लोन लिखना, म्यूटेबल फ़ील्ड आने पर उसे डीप क्लोन में अपग्रेड करना, फिर एक रजिस्ट्री और क्लाइंट कोड जोड़ना।

Step 1 — The Prototype Interface

The interface stays intentionally minimal. It says nothing about fields or how copying happens internally — that's entirely up to each concrete class.

इंटरफ़ेस जान-बूझकर न्यूनतम (minimal) रखा गया है। यह फ़ील्ड्स के बारे में या अंदर कॉपी कैसे होगी, इस बारे में कुछ नहीं कहता — यह पूरी तरह हर कॉन्क्रीट क्लास पर निर्भर करता है।

```
<?php

interface EnemyPrototype
{
    public function clone(): EnemyPrototype;
}
```

Step 2 — A Concrete Prototype with a Shallow Clone

At this stage every field is a primitive (int, float, string), so a shallow clone is correct and sufficient. The clone is a fully independent object — changing the copy's health never touches the original.

इस चरण पर हर फ़ील्ड एक प्रिमिटिव (int, float, string) है, इसलिए एक शैलो क्लोन सही और पर्याप्त है। क्लोन एक पूरी तरह स्वतंत्र ऑब्जेक्ट है — कॉपी की हेल्थ बदलने से मूल पर कभी असर नहीं पड़ता।

```

<?php

class Enemy implements EnemyPrototype
{
    public function __construct(
        public string $type,
        public int $health,
        public int $speed,
        public bool $armored,
        public string $weapon
    ) {
    }

    // Shallow clone: fine here because every field above is a primitive.
    // शैलो क्लोन: यहाँ ठीक है क्योंकि ऊपर हर फ़िल्ड एक प्रिमिटिव है।
    public function clone(): EnemyPrototype
    {
        return new self($this->type, $this->health, $this->speed, $this->armored,
            $this->weapon);
    }
}

```

Step 3 — Adding a Mutable Field Breaks the Shallow Clone (and How to Fix It)

Add an `inventory` array of items the enemy is carrying, and the shallow clone above becomes wrong: PHP arrays are copy-on-write for the array structure itself, but if the array holds *objects* (like `Item` instances below), those objects would still be shared between the original and the clone unless each one is cloned too. The fix is to deep-copy every mutable reference field inside `clone()`.

एक `inventory` array जोड़िए, जिसमें एनिमी के पास मौजूद आइटम्स हों, और ऊपर वाली शैलो क्लोन ग़लत हो जाती है: PHP arrays खुद array structure के लिए copy-on-write होते हैं, लेकिन अगर array में ऑब्जेक्ट्स (जैसे नीचे दिए गए `Item` instances) हों, तो वे ऑब्जेक्ट्स मूल और क्लोन के बीच साझा (shared) ही रहेंगे, जब तक हर एक को भी क्लोन न किया जाए। समाधान यह है कि `clone()` के अंदर हर म्यूटेबल रेफ़रेंस फ़िल्ड को डीप-कॉपी किया जाए।

```

<?php

class Item
{
    public function __construct(public string $name, public int $power)
    {
    }

    public function clone(): self
    {
        return new self($this->name, $this->power);
    }
}

class Enemy implements EnemyPrototype
{
    /** @var Item[] */
    private array $inventory = [];

    public function __construct(
        public string $type,
        public int $health,
        public int $speed,
        public bool $armored,
        public string $weapon
    ) {
    }

    public function addItem(Item $item): void
    {
        $this->inventory[] = $item;
    }

    /** @return Item[] */
    public function getInventory(): array
    {
        return $this->inventory;
    }

    // Deep clone: every Item object in $inventory is cloned individually,
    // so the clone's inventory list is fully independent of the original's.
    // डीप क्लोन: $inventory में मौजूद हर Item ऑब्जेक्ट को अलग-अलग क्लोन किया जाता है,
    // ताकि क्लोन की इन्वेंट्री लिस्ट मूल से पूरी तरह स्वतंत्र रहे।
    public function clone(): EnemyPrototype
    {
        $copy = new self($this->type, $this->health, $this->speed, $this->armored,
$this->weapon);
        foreach ($this->inventory as $item) {
            $copy->addItem($item->clone());
        }
        return $copy;
    }
}

```

Step 4 — The Prototype Registry

The registry stores one configured prototype per enemy type and always hands back a clone, never the stored original — if it ever returned the original, a caller mutating "their" enemy would silently corrupt every future clone.

रजिस्ट्री हर एनिमी टाइप के लिए एक कॉन्फिगर किया प्रोटोटाइप स्टोर करती है और हमेशा एक क्लोन लौटाती है, कभी भी स्टोर किया मूल नहीं — अगर यह कभी मूल लौटा दे, तो "अपने" एनिमी में बदलाव करने वाला कोई कॉलर चुपचाप हर आगे के क्लोन को खराब कर देगा।

```
<?php

class EnemyRegistry
{
    /** @var array<string, EnemyPrototype> */
    private array $prototypes = [];

    public function register(string $key, EnemyPrototype $prototype): void
    {
        $this->prototypes[$key] = $prototype;
    }

    public function get(string $key): EnemyPrototype
    {
        if (!isset($this->prototypes[$key])) {
            throw new \InvalidArgumentException("No prototype registered for key:
{$key}");
        }
        // Always return a clone – the registry's own copy must never be
        // handed out and never be mutated by a caller.
        // हमेशा एक क्लोन लौटाएँ – रजिस्ट्री की अपनी कॉपी कभी बाहर नहीं दी जानी चाहिए
        // और कभी किसी कॉलर द्वारा बदली नहीं जानी चाहिए।
        return $this->prototypes[$key]->clone();
    }
}
```

Step 5 — Client Code

```
<?php

// Configure each prototype once. / हर प्रोटोटाइप को एक ही बार कॉन्फ़िगर करें।
$flying = new Enemy(type: "Flying", health: 60, speed: 9, armored: false, weapon:
"Laser");
$flying->addItem(new Item("Speed Boost", 5));

$armored = new Enemy(type: "Armored", health: 150, speed: 3, armored: true, weapon:
"Cannon");
$armored->addItem(new Item("Shield Plate", 10));

$registry = new EnemyRegistry();
$registry->register("flying", $flying);
$registry->register("armored", $armored);

// Spawn enemies purely by cloning – never by calling `new Enemy(...)` again.
// एनिमीज़ को सिर्फ़ क्लोनिंग से स्पॉन करें – दोबारा `new Enemy(...)` कॉल किए बिना।
$enemy1 = $registry->get("flying");
$enemy2 = $registry->get("flying");

// Customize one clone without touching the original prototype or the other clone.
// एक क्लोन को कस्टमाइज़ करें, बिना मूल प्रोटोटाइप या दूसरे क्लोन को छुए।
$enemy2->health = 30; // a "weakened" flying enemy / एक "कमज़ोर" फ़्लाइंग एनिमी

printf("Enemy 1: %s, health=%d, inventory=%d item(s)\n", $enemy1->type, $enemy1-
>health, count($enemy1->getInventory()));
printf("Enemy 2: %s, health=%d, inventory=%d item(s)\n", $enemy2->type, $enemy2-
>health, count($enemy2->getInventory()));
printf("Original prototype health untouched: %d\n", $flying->health);
```

Expected output / अपेक्षित आउटपुट:

```
Enemy 1: Flying, health=60, inventory=1 item(s)
Enemy 2: Flying, health=30, inventory=1 item(s)
Original prototype health untouched: 60
```

6. Practical Example: Email Templates

Here's a second, unrelated scenario that makes the same point. Say you run a bulk-email system: one monthly newsletter body is shared company-wide, but each department needs its own subject line and its own recipient list. Rather than duplicating the whole email-construction call per department, you configure one base template once and clone it for each variant.

यहाँ एक दूसरा, बिल्कुल अलग परिदृश्य है जो वही बात साबित करता है। मान लीजिए आप एक बल्क-ईमेल (bulk-email) सिस्टम चलाते हैं: एक महीने का न्यूज़लेटर (newsletter) बॉडी पूरी कंपनी में साझा (shared) है, लेकिन हर डिपार्टमेंट को अपना खुद का सब्जेक्ट लाइन (subject line) और अपनी खुद की रिसिपिएंट लिस्ट (recipient list) चाहिए। हर डिपार्टमेंट के लिए पूरा ईमेल-कंस्ट्रक्शन कॉल दोहराने के बजाय, आप एक बेस टेम्पलेट (base template) एक बार कॉन्फ़िगर करते हैं और हर वैरिएंट (variant) के लिए उसे क्लोन करते हैं।

The catch is the same one as before: the recipient list is a nested, mutable object holding two lists — `to` and `cc`. A shallow clone would leave every department's email pointing at the *same* recipient list, so adding a recipient for Marketing would silently add them to HR and Engineering too. The fix, again, is a deep copy of that nested object.

पेच वही है जो पहले था: recipient list एक नेस्टेड, म्यूटेबल ऑब्जेक्ट है जिसमें दो लिस्ट्स होती हैं — `to` और `cc`। एक शैलो क्लोन हर डिपार्टमेंट के ईमेल को उसी recipient list की ओर इशारा करता छोड़ देगा, इसलिए Marketing के लिए एक recipient जोड़ने से चुपचाप वह HR और Engineering में भी जुड़ जाएगा। समाधान, फिर से, उस नेस्टेड ऑब्जेक्ट की एक डीप कॉपी है।

```

<?php

class RecipientList
{
    /** @var string[] */
    private array $to;

    /** @var string[] */
    private array $cc;

    public function __construct(array $to = [], array $cc = [])
    {
        $this->to = $to;
        $this->cc = $cc;
    }

    public function addTo(string $email): void
    {
        $this->to[] = $email;
    }

    public function addCc(string $email): void
    {
        $this->cc[] = $email;
    }

    public function summary(): string
    {
        return sprintf("to=[%s], cc=[%s]", implode(", ", $this->to), implode(", ",
$this->cc));
    }

    // Deep copy: new arrays, so mutating the copy's lists never touches the
    original's.
    // डीप कॉपी: नई arrays, ताकि कॉपी की लिस्ट्स बदलने से मूल पर कभी असर न पड़े।
    public function deepCopy(): self
    {
        return new self($this->to, $this->cc);
    }
}

class EmailTemplate
{
    public function __construct(
        public string $subject,
        public string $body,
        private RecipientList $recipients
    ) {
    }

    public function addRecipient(string $email): void
    {
        $this->recipients->addTo($email);
    }

    public function addCc(string $email): void
    {
        $this->recipients->addCc($email);
    }
}

```



```

    }

    public function recipientSummary(): string
    {
        return $this->recipients->summary();
    }

    // clone() calls recipients->deepCopy() – without this line, every
    // department clone would share and corrupt the same recipient list.
    // clone() में recipients->deepCopy() कॉल होता है – इस लाइन के बिना, हर
    // डिपार्टमेंट का क्लोन एक ही recipient list साझा करके उसे बिगाड़ देगा।
    public function clone(): self
    {
        return new self($this->subject, $this->body, $this->recipients->deepCopy());
    }
}

// Base template configured once. / बेस टेम्पलेट एक ही बार कॉन्फ़िगर किया गया।
$base = new EmailTemplate(
    subject: "Monthly Newsletter",
    body: "Here's what happened across the company this month...",
    recipients: new RecipientList(to: ["all-staff@company.com"])
);

$hr = $base->clone();
$hr->subject = "Monthly Newsletter – HR Edition";
$hr->addRecipient("hr-team@company.com");
$hr->addCc("hr-lead@company.com");

$marketing = $base->clone();
$marketing->subject = "Monthly Newsletter – Marketing Edition";
$marketing->addRecipient("marketing-team@company.com");

$engineering = $base->clone();
$engineering->subject = "Monthly Newsletter – Engineering Edition";
$engineering->addRecipient("engineering-team@company.com");

printf("Base:          %s\n", $base->recipientSummary());
printf("HR:           %s\n", $hr->recipientSummary());
printf("Marketing:    %s\n", $marketing->recipientSummary());
printf("Engineering: %s\n", $engineering->recipientSummary());

```

Expected output / अपेक्षित आउटपुट:

```

Base:          to=[all-staff@company.com], cc=[]
HR:           to=[all-staff@company.com, hr-team@company.com], cc=[hr-lead@company.com]
Marketing:    to=[all-staff@company.com, marketing-team@company.com], cc=[]
Engineering: to=[all-staff@company.com, engineering-team@company.com], cc=[]

```

This works precisely because `clone()` calls `$this->recipients->deepCopy()` instead of just handing the same `RecipientList` object to all three departments. Without that one call, adding a recipient to Marketing's email would have silently leaked into HR's and Engineering's recipient lists too, and even into the base template itself.

यह ठीक इसलिए काम करता है क्योंकि `clone()` , तीनों डिपार्टमेंट्स को एक ही `RecipientList` ऑब्जेक्ट सौंपने के बजाय, `$this->recipients->deepCopy()` कॉल करता है। इस एक कॉल के बिना, Marketing के ईमेल में एक recipient जोड़ने से वह चुपचाप HR और Engineering की recipient lists में भी, और यहाँ तक कि बेस टेम्पलेट में भी, लीक (leak) हो जाता।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Prototype, Factory और Builder — तीनों क्रिएशनल पैटर्न हैं, जो ऑब्जेक्ट बनाने से जुड़े हैं।
Prototype	प्रोटोटाइप	<code>\$flying</code> एक प्रोटोटाइप है — <code>\$registry->get("flying")</code> कॉल करने पर इसका एक क्लोन मिलता है।
Clone (verb/ method)	क्लोन करना / <code>clone()</code> मेथड	<code>\$enemy2 = \$registry->get("flying");</code> अंदर से <code>clone()</code> कॉल करता है।
Shallow Copy	शैलो कॉपी	<code>Enemy</code> क्लास का पहला वर्शन शैलो कॉपी करता था — सिर्फ़ प्रिमिटिव फ़िल्ड्स के लिए सही।
Deep Copy	डीप कॉपी	<code>RecipientList::deepCopy()</code> to और <code>cc</code> की नई arrays बनाकर एक डीप कॉपी करता है।
Mutable	म्यूटेबल (बदलने योग्य)	<code>inventory</code> array और <code>RecipientList</code> दोनों म्यूटेबल हैं — इन्हें डीप-कॉपी करना ज़रूरी है।
Immutable	इम्यूटेबल (न बदलने योग्य)	<code>string</code> , <code>int</code> जैसी प्रिमिटिव वैल्यूज़ इम्यूटेबल की तरह व्यवहार करती हैं — कॉपी करना सुरक्षित है।
Encapsulation	एनकैप्सुलेशन	<code>Enemy</code> की <code>private</code> array <code>\$inventory</code> फ़िल्ड, क्लास के बाहर से सीधे नहीं बदली जा सकती।
Open/Closed Principle	ओपन/क्लोज़्ड प्रिंसिपल	रजिस्ट्री में एक नया एनिमी टाइप जोड़ना मौजूदा कोड बदले बिना संभव है — यही OCP है।
Interface	इंटरफ़ेस	<code>EnemyPrototype</code> इंटरफ़ेस सिर्फ़ <code>clone()</code> मेथड डिक्लेयर करता है।
Concrete Class	कॉन्क्रीट क्लास (असली क्लास)	<code>Enemy</code> , <code>EnemyPrototype</code> इंटरफ़ेस की एक कॉन्क्रीट क्लास है।
Registry	रजिस्ट्री	<code>EnemyRegistry</code> , <code>"flying" => \$flyingPrototype</code> जैसी एंट्रीज़ स्टोर करती है।
Client Code	क्लाइंट कोड	वह कोड जो <code>\$registry->get("flying")</code> कॉल करता है, वह क्लाइंट कोड है।
Polymorphism	पॉलिमॉर्फ़िज़्म	<code>EnemyPrototype</code> टाइप के किसी भी ऑब्जेक्ट पर <code>clone()</code> कॉल करने पर, सही असली क्लास का <code>clone()</code> ही चलता है।
Boilerplate	बॉयलरप्लेट (दोहराया जाने वाला कोड)	हर एनिमी के लिए पूरा कंस्ट्रक्टर कॉल दोहराना बॉयलरप्लेट कोड है, जिसे Prototype कम कर देता है।
Nested Object	नेस्टेड ऑब्जेक्ट	<code>EmailTemplate</code> के अंदर मौजूद <code>RecipientList</code> एक नेस्टेड ऑब्जेक्ट है।

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Copy Constructor (pattern)	कॉपी कंस्ट्रक्टर (पैटर्न)	<code>new self(\$this->type, \$this->health, ...)</code> — मूल ऑब्जेक्ट की वैल्यूज़ पास करके एक नया instance बनाना।
Prototype Registry	प्रोटोटाइप रजिस्ट्री	<code>EnemyRegistry</code> क्लास, key के आधार पर प्रोटोटाइप्स ढूँढ़ने और क्लोन करने का काम करती है।

General Words Glossary / सामान्य शब्दों की शब्दावली

Beyond the technical terms above, this document also uses everyday English words that a Hindi-primary reader may not know. Each one below is defined with its Hindi meaning and a plain, non-technical example sentence.

ऊपर दिए तकनीकी शब्दों के अलावा, इस दस्तावेज़ में कुछ सामान्य (everyday) अंग्रेज़ी शब्द भी इस्तेमाल हुए हैं, जो हिंदी-प्रधान (Hindi-primary) पाठक को अपरिचित लग सकते हैं। नीचे हर शब्द का हिंदी अर्थ और एक सामान्य (non-technical) उदाहरण वाक्य दिया गया है।

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Instinct	सहज प्रवृत्ति, पहला खयाल	"Her first instinct was to call her mother when she heard the news." खबर सुनते ही उसका पहला खयाल अपनी माँ को फ़ोन करने का था।
Scattered	बिखरा हुआ	"Toys were scattered across the living room floor." खिलौने पूरे लिविंग रूम के फ़र्श पर बिखरे हुए थे।
Hunting (for something)	ढूँढ़ना, खोजबीन करना	"He spent the whole evening hunting for his lost keys." उसने पूरी शाम अपनी खोई हुई चाबियाँ ढूँढ़ने में बिताई।
Stuck	अटक जाना, फँस जाना	"The car got stuck in the mud after the rain." बारिश के बाद गाड़ी कीचड़ में फँस गई।
Brittle	नाज़ुक, आसानी से टूटने वाला	"Old newspapers become brittle and tear easily." पुराने अख़बार नाज़ुक हो जाते हैं और आसानी से फट जाते हैं।
Awkward	अजीब, असहज	"There was an awkward silence after his joke fell flat." उसका मज़ाक न चलने के बाद एक अजीब-सी खामोशी छा गई।
Catch (a catch / the catch)	पेच, छिपी हुई अड़चन	"The offer seemed too good — there had to be a catch." यह ऑफ़र बहुत अच्छा लग रहा था — कहीं न कहीं कोई पेच ज़रूर होगा।
Obvious	स्पष्ट, साफ़	"It was obvious from her smile that she was happy." उसकी मुस्कान से साफ़ था कि वह खुश थी।
Tweak / Tweaked	थोड़ा बदलाव करना	"He tweaked the recipe slightly by adding more salt." उसने थोड़ा और नमक डालकर रेसिपी में मामूली बदलाव किया।
Consistent	एक जैसा, संगत, स्थिर	"She's been consistent with her morning walks for a year." वह एक साल से लगातार एक जैसे समय पर सुबह की सैर कर रही है।
Cuts down (on something)	घटाना, कम करना	"Meal-prepping on Sundays cuts down on weekday cooking time." रविवार को भोजन पहले से तैयार करने से हफ़्ते के दिनों में खाना बनाने का समय घट जाता है।
Corrupt(ed)	खराब, बिगड़ा हुआ	"The file got corrupted when the laptop shut down suddenly." लैपटॉप अचानक बंद होने से फ़ाइल खराब हो गई।
Leak(ed)	रिसना, लीक होना	

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
		"News of the merger leaked before the official announcement." आधिकारिक घोषणा से पहले ही विलय (merger) की खबर लीक हो गई।
Untouched	अछूता, बिना छेड़े	"The dessert on his plate remained untouched all evening." उसकी थाली में रखी मिठाई पूरी शाम अछूती रही।
Earns its keep	अपनी उपयोगिता साबित करना, अपने होने को सही ठहराना	"That old tool still earns its keep every time we renovate a room." वह पुराना औज़ार आज भी अपनी उपयोगिता साबित करता है, जब भी हम कोई कमरा नवीनीकृत करते हैं।
Silently	चुपचाप	"He silently left the room so as not to wake the baby." बच्चे को जगाने से बचने के लिए वह चुपचाप कमरे से निकल गया।
Deliberately	जान-बूझकर	"She deliberately arrived early to get a good seat." अच्छी सीट पाने के लिए वह जान-बूझकर जल्दी पहुँची।
Genuinely	वाक़ई, सचमुच	"He was genuinely surprised by the birthday party." वह जन्मदिन की पार्टी से वाक़ई हैरान था।
Reliable / Reliably	भरोसेमंद	"This bus service has always been reliable." यह बस सेवा हमेशा भरोसेमंद रही है।
Baseline	आधार रेखा, बुनियादी स्तर	"We measured everyone's fitness as a baseline before starting the program." प्रोग्राम शुरू करने से पहले हमने सभी की फ़िटनेस को आधार रेखा के तौर पर मापा।
Brand-new	बिल्कुल नया	"He was excited to drive his brand-new car for the first time." वह अपनी बिल्कुल नई गाड़ी पहली बार चलाने को लेकर उत्साहित था।

This document follows the topic order of [algomaster.io](https://www.algomaster.io)'s Prototype lesson (a paid course) but is written independently — original English/Hindi explanations and original PHP code — rather than reproducing their (subscription-gated) material.

यह दस्तावेज़ [algomaster.io](https://www.algomaster.io) के Prototype पाठ (एक पेड कोर्स) के विषय-क्रम को फ़ॉलो करता है, लेकिन स्वतंत्र रूप से लिखा गया है — ओरिजिनल अंग्रेज़ी/हिंदी व्याख्याएँ और ओरिजिनल PHP कोड — बजाय इसके कि उनकी (सब्सक्रिप्शन-गेटेड) सामग्री को दोबारा प्रस्तुत किया जाए।